

4th International Conference on Innovative Data Communication Technology and Application

mURLi: A Tool for Detection of Malicious URLs and Injection Attacks

Vihar Devalla^a, Srinivasa Raghavan S^b, Swati Maste^c, Jaaswin D Kotian^d, Dr. Annapurna D^e

^aDept. of Computer Science Engineering, PES University, Bengaluru, Karnataka 560100, India

^bDept. of Computer Science Engineering, PES University, Bengaluru, Karnataka 560100, India

^cDept. of Computer Science Engineering, PES University, Bengaluru, Karnataka 560100, India

^dDept. of Computer Science Engineering, PES University, Bengaluru, Karnataka 560100, India

^eProfessor, Dept. of Computer Science Engineering, PES University, Bengaluru, Karnataka 560100, India

Abstract

SQL databases have been a staple for storing data in enterprises to meet their database requirements. SQL injection has also become a major security threat due to the growth of SQL. Injection attacks occur when the query in the database is modified by malicious data from an unsecure user. The use of NoSQL databases has been growing steadily due to advancements in Big Data and similar domains, and as an alternative to SQL databases. But recent studies have proved that even NoSQL is susceptible to injection attacks. Malicious websites can harm a person using the internet in various ways. Phishing attacks, Spam attacks, Defacement Attacks can occur once a user clicks on a URL. In this research, we propose the tool mURLi, for early detection of Malicious URLs, SQLi and NoSQLi, which pose a large threat to the end-users. To build mURLi, we have used various deep learning models (ANN, BiLSTM, BERT) and machine learning models (Random Forest, XGBoost, Adaboost, KNN) in an attempt to detect these attacks (SQLi, NoSQLi and Malicious URLs) more accurately than traditional methods and have presented a comparative study of the same. The results show that BERT model outperforms the existing tools for SQL and NoSQL attacks with an accuracy of 99.6% and 99.01% respectively, which was achievable due to its pre-training and deep language knowledge. BiLSTM proves to be the best tool for detection of Malicious URL attacks, with the results being 95.2% accurate.

© 2023 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 4th International Conference on Innovative Data Communication Technologies and Application

Keywords: mURLi (Malicious URL and Injection Attacks Detection), SQL Injection (SQLi), NoSQL Injection (NoSQLi), Phishing, Spam, Malware, Machine Learning (ML), Deep Learning (DL), Random Forest (RF), XGBoost, Adaboost, KNN (K-Nearest Neighbour), Artificial Neural Network (ANN), Bidirectional Long Short Term Memory (BiLSTM), Bidirectional Encoder Representations from Transformers (BERT)

E-mail address: sraghavsrg@gmail.com

1. Introduction

1.1. Injection Attacks

SQL databases are relational databases which have a fixed schema and store structured data. The data for these databases is stored in tables. Tables contain a collection of rows, with each attribute called a column. SQL databases are still one of the most used databases, especially in enterprises and legacy applications. Some popular SQL clients used for hosting a SQL database are Oracle SQL, MySQL, Microsoft SQL Server and PostgreSQL. SQL databases have a fatal flaw which also happens to be one of the oldest database vulnerability to exist. An SQL Injection attack takes place when a malicious entity manages to modify a SQL query using invalid/unverified input. There have been various security incidents involving SQL injection. CSO Online [1] talks about an attack on Heartland Payment Systems in 2009 which used SQL – Injection attack and attackers stole more than 130 million credit and debit card details. This attack vector still persists, for example the attack on the organizations, Freepik and Flaticon in 2020 where hackers were able to steal email and password details of 8.3 million users, which used a SQL Injection vulnerability to initiate the attack [2].

NoSQL (Not Only SQL) databases are non-relational databases which store data in documents instead of tables. They have a flexible schema, can hold unstructured data and scale flexibly. [3] They are an alternative to SQL databases and used in cloud services which have a huge volume of data to store and query. But NoSQL databases are also vulnerable to injection attacks. The attack on Rocket.Chat in 2021 allowed an attacker to remotely execute commands on a victim machine using a NoSQL injection attack [4]. Similarly, an attack on the Iranian social media app Raychat exposed 150 million users' personal data and records online. This attack too is stemmed from an insecure NoSQL database[5].

All these attacks show that a basic countermeasure like input sanitation is not enough in the current technological landscape. In this research, we present different methods to detect SQL Injection and NoSQL injection in queries using machine learning models like Random Forest, XGBoost, Adaboost, KNN and deep learning models like ANN, BiLSTM, BERT.

1.2. Malicious Websites

The World Wide Web is a vast, open network with many users and applications. But it also allows adversaries to execute cyberattacks and trap victims. Malicious URLs could be used to trick the user to cause financial and personal data loss. Some attacks used to manipulate the users include Phishing Attacks, Spam Attacks and Malware links. All these attacks do their bidding when the victim accesses an unverified/malicious URL without his knowledge.

1.2.1. Phishing

This attack involves obtaining sensitive and confidential information about a user by posing as a legal authority or organization.

1.2.2. Spam

The sending of unsolicited messages to numerous user with malicious intent is called spamming.

1.2.3. Malware

The act of installing malicious software in the victim's devices.

Malicious URLs have been traditionally blocked by creating a blacklist and avoiding such links. But with the advent of proxy technologies, this countermeasure doesn't hold up very well on its own.

In our research, we propose the use of advanced deep learning models like ANN and BiLSTM and extensive feature extraction before training the machine learning models.

2. Related Works

Data is the most precious resource in present times, that it is being referred to as “fuel” that powers the contemporary world. Ever since data has gained popularity, it has seen an increase in hacker's activities, indicating security

threats. Even though there exist ways to protect the sensitive data, theft of such data still prevails and is catastrophic.

NoSQL is not only capable of handling unstructured data but has immensely supported the trend of big data and hence has gained popularity in recent times. The accelerating usage of this technology in current applications brings with it the security concerns to the surface. Islam, Md Rafid Ul, et al. proposed a tool for automatic detection of NoSQL injections[20]. They have created their own training dataset and have manually developed essential features and then applied different supervised learning algorithms on it. This tool achieved impressive results and outperformed the prevailing tool Sqreen.

The existence of a very small dataset made it difficult to train the model, hence they used data augmentation techniques like, crossovers and mutations, to generate a good number of safe and malicious queries. This dataset has both MongoDB and CouchDB queries and includes 4 types of injections which has JavaScript-based and piggybacked queries, which Sqreen tools fail to detect. As this dataset is imbalanced, SMOTE (synthetic minority oversampling technique) was used to create the oversamples of the lesser records and help achieve a balanced dataset.

After creating the labelled dataset, the authors went on to manually design and develop essential features, of which the top 10 were ranked and selected using the ID3 algorithm. They trained a variety of models on this dataset, such as, random forest, ANN with back propagation, KNN and XGBoost [11] [12] [13] [14] [15].

The results proved that their proposed tool outperforms the Sqreen tool and is also database-agnostic, as it achieved similar results with MongoDB and CouchDB.

Shachi, Mehjabeen, et al. worked on and briefed the various security threats experienced by both SQL and NoSQL databases. They provide an overview of the prevailing techniques in order to detect and prevent various SQL and NoSQL injection attacks, which are ML based and non-ML based approaches [21]. The former approach used supervised learning models like SVM [22], KNN and ensemble learners like AdaBoost and XGBoost and have been trained for a classification task [11] [12] [13] [14] [15]. To improvise on the model performance, the features were ranked using a decision tree [10] and only the top ten features were sent to the model for training[9] [27] [28]. The latter approach has a variety of techniques for the task at hand like, building an automation and using it for the detection task [16], to, different analysis methods (Static/Dynamic/Combined approach)[23][24][25][26].

Server-side request forgery (SSRF) is a security loophole that exists due to susceptible internet applications [18] [19]. Al-talak, Khadejah et al. in their research briefed about the different types of SSRF attacks [37] and also discusses the preventive measures against these attacks. The prevailing work has extensively made use of wide-range of machine learning techniques but, this paper uses deep learning approach (LSTM) to create a clever model that is capable of detecting such attacks. Among the various pre-processing techniques used, the one that helped in identifying the data patterns was scaling the data to range from -1 to +1. They modelled this scaled data using LSTM, which is especially designed to learn and remember the long-term dependencies and overcome the issues with RNN [17]. This model was later used to detect attacks in user requests that help to decide whether to process this request or to deny the service based on the existence of the attack. The future scope of this work is to use better pre-processing methods and experiment with modelling this data with various other deep learning techniques.

The key take-away from this survey paper was to, either improvise on the existing methodology or to try out deep learning models and compare to see the best suitable approach for the detection task. Motivated by their work, we have carried out our research.

3. Methodology

We propose our tool mURLi (Malicious URL and Injection Attacks Detection) where we have used various intelligent models to figure out the most effective model to detect these attacks. The figure below shows our proposed detailed design that we followed to perform our experiment.

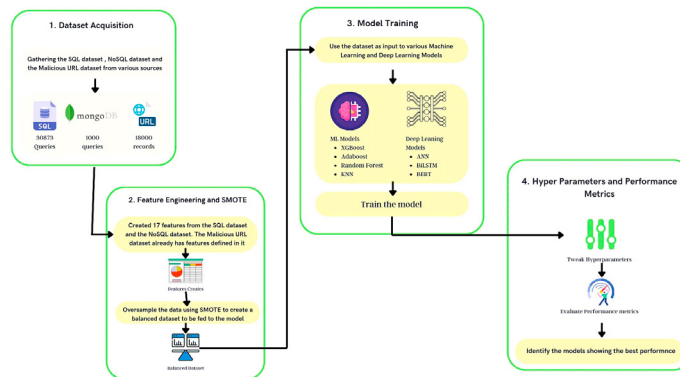


Fig. 1. Proposed Detailed design

3.1. Dataset

The intelligent models, we used, require a dataset to train and understand the features of the data. To achieve this, we found a couple of datasets containing SQL Injection, NoSQL Injection and Malicious URL data points. The SQL dataset was found on Kaggle, which contained about, 30873 different SQL queries labelled as benign or vulnerable. The attributes in the dataset are :

1. Sentences – This attribute holds the SQL query used, which can be vulnerable or safe.
2. Label – This attribute conveys if the Sentence, i.e., the SQL query is safe or vulnerable class, with text defining these states.

Similarly, the NoSQL database taken from [5] which contains benign and injectable NoSQL queries in text files which are converted to a Pandas Data frame using Python. The initial text files had the query in a single line, which was parsed in Pandas and converted into the Sentence and Label format as seen in the SQL dataset. The number of MongoDB queries used are 1000 data points with 800 benign and 200 vulnerable queries.

The Malicious URL dataset is taken from the Canadian Institute for Cybersecurity of University of New Brunswick, the dataset is called the ISCX-URL2016 as illustrated in [6] which includes its creation and analysis. This dataset contains 18 thousand records with 80 attributes.

3.2. Preprocessing

The most common step during preprocessing is to check for NULL values or missing data in our dataset. As we have a good number of records to train our model, we did not fill the missing values using mean/mode instead, we removed all the null and missing value records using Pandas Python Library. After this step, we then move to other necessary steps to achieve a clean dataset. The values of target-label in injection attacks dataset are categorical and hence, it is necessary to convert them into numerical value. To do so, we assign "safe queries" as 0 and "vulnerable queries" as 1. This creates a comprehensive dataset that can be used to train the intelligent models.

Another approach to handle Text/categorical data is by encoding. Categorical datasets are very common in real life, from collecting data in 'yes' and 'no' to describing data in sentences, text data is used at a large scale in machine learning.

However, our machine learning models can only read and comprehend numerical values. Hence, it is essential to encode categorical features to numeric values. We have used One Hot Encoder to convert categorical data to numeric values. One Hot Encoder assigns binary values and creates new columns depending on the number of class labels present.

Another challenge faced is the presence of Imbalanced datasets. Such datasets are hard to work with, as the predictive models developed using them for their training, have poor performance on the minority class of the target classes. The technique used to address this problem is to over sample the minority class by duplication of existing training data. We use SMOTE or Synthetic Minority Oversampling Technique to balance the dataset. SMOTE generates the synthetic samples for the minority class, and this helps us achieve a balanced dataset such that each target class have an equal number of training records. This is crucial as it helps to avoid bias during model training. Using SMOTE, we successfully balanced all the datasets used for training. Addressing the skewness of training dataset highly impacts the model performance, as seen in the results during this research.

3.3. Feature Engineering

1. **Malicious URL** - The URLs data has been collected and preprocessed and converted to dataset containing 80 columns and 18k records. Each record contains variety of details like URL length, isPortEighty, Extension, Digitalcount etc. The feature engineering for the same has been explained in [6].
2. **SQL and NoSQL Dataset** – Machine Learning Models use the various features available in the dataset to train itself. The dataset before feature engineering only has 1 feature, that being the query itself. We have created new features for the dataset following the learnings in [20] and proposed a few extensions and created the features. Listed below are the features using the query as the base.
 - (a) **Empty String in Query** : Empty Strings can be used in SQL and NoSQL queries to create an always true statement and ignore the logic for the query.
 - (b) **Comparison Operators in Query** : We use a regex-like format to check for usage of comparison operators used in the NoSQL query like "find()", "find.sort()", "\$eq", "\$gt", "\$gte", "\$lt", "\$lte", "\$ne", "\$in", "\$nin".
 - (c) **Contains Payload from WordLists**: The attack could be done using a payload file, which might have a list of vulnerable SQL or NoSQL queries. We use the wordlist from "nosqlinjection wordlists" repository from cr0hn [29] and check if the query contains any part which resembles the wordlist payloads, similarly SQL payloads are also found[36].
 - (d) **Not Equal Operator in Query**: 'ne' keyword is used to denote the not equal operator in SQL and NoSQL query. It can be injected by an attacker to manipulate the query
 - (e) **Logical Operators in Query** : Logical Operators can be injected to modify the logic of the query, some of them include "\$or", "\$and", "\$not", "all", "and", "or"
 - (f) **Evaluation Query Operation** : We check for queries which have evaluation queries like "\$where", "\$mod", "\$regex", "\$text", "\$where".
 - (g) **Return Statement in Query** : Return Statements can be injected to close the query and return unauthorized data. The regex searches for ";return", "return true", "return 1", "return" in the query
 - (h) **Always True Statements** : "/./", "/.*/", "TRUE", "1", "=", "true" commands return true for every entry in query as it is an "any" query.
 - (i) **Element Checking Query** : We search for "\$exists", "\$type" which is used to find elements separately and could be used for attacking MongoDB databases.
 - (j) **Null Keyword in Query** : Any comparison with null keyword always returns true, so invoking it could be a risk for the query.
 - (k) **Table Query** : We search for queries which target the table operations, like "createTable()", etc.
 - (l) **Drop Data** : Dropping tables or databases can lead to huge data loss for a given application. We search for "drop", "database" keywords in the query
 - (m) **Remove Operation in Query** : "remove" keyword can be used to delete information in the NoSQL database.
 - (n) **Limit Query** : Limit query is used to reduce the number of rows sent as output for the query, manipulating this could lead to access to unauthorized data.
 - (o) **Infinite Loop and Functions** : Injection functions and loops can lead to unavailability and manipulation of data. We search for "while(true)", "\$in", "for", "do", "()"=; in the database.
 - (p) **Comments in Query** : Comments can be used to stop queries defined in server and injecting malicious query instead.

- (q) **Check for Matching, Finding and similar queries** : These queries are used to find particular elements in the database which include "match","find","search".

We captured these features from each query in our dataset and encoded them as '0' and '1', where zero indicates the absence of that feature and one indicates the presence of a feature. This was done individually to extract each of the above listed features and are now the features columns in our dataset, which is used for model training.

3.4. Analysis of Models used

We experimented with the following machine learning(RF, KNN, XGBoost, AdaBoost) and deep learning models(ANN, BiLSTM, BERT), one of the key reasons to proceed with these models is their impressive performance on various tasks and flexibility of hyperparameter tuning. It led us to carry out the research to understand which models are the best suitable for our problem statement. This section talks in detail about each model and their respective hyperparameters used during this experiment.

3.4.1. Random Forest

Random Decision Forests[38] is one of the ensemble learning algorithm and is a supervised learning model. One of the most peculiar characteristics of this algorithm is that it works well with both categorical and real values. It implicitly makes use of the bagging method to group the decision trees. It is an ensemble learner, which is a learning technique of enhancing the model's performance by combining numerous classifiers to solve a complicated issue. Random Forest is diverse in nature since it doesn't account for all the attributes and features while building each tree. Every tree has of its own data and features and thus makes complete usage of the CPU while building random forests as whole. Also, there is no need of splitting the data into training and testing sets, since there is always 30% of data that is unseen by the decision tree.

Hyperparameters contribute majorly in model's learning process and with the use of optimal hyperparameters the model achieves the best performance. Hyperparameters used in the model:

- **n_estimators**: Number of trees the algorithm builds before averaging. The value of n_estimators in our experiment is 5.
- **max_features** : maximum number of features considered while node splitting. We used the default value, which is 'sqrt(n_features)'.
- **min_sample_leaf**: minimum number of leaves required while internal node splitting which is our experiment is 1.
- **criterion**: It is the function to measure the quality of a split, we used 'gini index' as criteria to measure the impurity.

3.4.2. K Nearest Neighbour

K-Nearest Neighbour[8][39] algorithm is one of the most widely used machine learning algorithms, which is yet another supervised learner. This method makes a key assumption that the unseen data and neighbour are related, and places this new data in the class that is very alike among the existing classes. This means that any unknown data points can be easily classified into a one of the existing category, using some distance measure to estimate the similarity between the test sample and the target classes to make the classification decision. It's a non-parametric model, but can also be applied for both regression and classification tasks. KNN is a lazy learner and is sometimes called as instance-based technique, as it essentially relies on memory to maintain the training data. In order to find the nearest neighbours of each query point it makes use of distance metrics such as Euclidean distance, Manhattan distance, Minkowski distance, Hamming distance and many others. Euclidean distance is the most widely used distance measure, and we have used it as a metric as part of this research work while training the KNN model for the classification task.

Euclidean distance: This is a very popular and widely used distance measure and is the default metric that SKlearn library of Python uses for K-Nearest Neighbour. It computes a straight line between the query point and the known data points. It is calculated as the square root of the sum of the squared differences between the two vectors. Let 'q' be the query point and 'x' be each training sample, then the euclidean distance can be computed using the formula given below:

$$d(q, x) = \sqrt{\sum_{i=1}^n (x_i - q_i)^2}$$

The hyperparameters for KNN used in our experiment are:

- leaf_size: Leaf size passed to BallTree or KDTree. The value of leaf_size in our experiment is 13.
- p : p is the Power parameter for the Minkowski metric. We have used p = 1, this is equivalent to using Manhattan distance.
- n_neighbors: Number of neighbours to use by default for 'k' neighbours queries. The value for nearest neighbours we used is 7.

3.4.3. XGBoost

XGBoost[30][40] is an abbreviation of Extreme Gradient Boosting. It is an implementation of gradient boosting-based decision tree, which is an ensemble learner. The key idea of this algorithm is that, each predictor corrects its predecessor's error. This algorithm consists of a unique combination of software and hardware fine-tuning approaches that provide outstanding results while utilizing less computing power. A variety of hyperparameters are included in the XGBoost implementation. Tuning these hyperparameters can improve results depending on the job at hand. In our research work, we have used the default hyperparameters for XGBoost as provided by the SciKit learn module, because the model already performed well during both training and testing.

3.4.4. AdaBoost

Boosting algorithms[41][42] are ensemble methods used to improve the prediction power by converting the weak learners into strong learners. Boosting is used in supervised learning for reducing the bias and reducing the variance, and works on the principle of learners growing sequentially. Adaboost, is an abbreviation for adaptive boosting, which one of the most popular ensemble learner and is mostly used with decision trees of one level or 1 split. This algorithm builds a learner and assigns equal weights to all the data points initially and eventually assigns higher weights to the samples after each iteration, such that it gives more importance to the higher weights in the next model. This process is continued until a lower error is received. In our research work, we have used the default hyperparameters for AdaBoost as provided by the SciKit learn module, because the model already performed well during both training and testing.

3.4.5. Artificial Neural Networks(ANN)

The ANNs[43] are used for a variety of problems and are based on a supervised approach which consists of three layers : input, hidden, output. They take inspiration from brain and nervous system and try modelling this complex structure as a connected network spread across the three layers mentioned above. ANN models, essentially, try to mimic the electrical activity of the nervous system. Typically, the neurons are connected to each other in such a way that the result of the current layer is fed as input to the consecutive layer. A neuron may or may not be connected to all the neurons of the subsequent layer. The input values to a processing element are multiplied by a weight $W(n, m)$ that coordinates the strengths of the neural pathways. All the input weights to a processing element are then aggregated

using an activation function, producing a single output value given as an input to the next layer neuron. The use of activation function is essential as it helps to decide which neurons need to be actuated and which need not, for the training process. We used "sigmoid" and "tanh" functions as activation functions in our model training. The course of action is followed across layers until the network produces an end result as a single value, or as a vector of values.

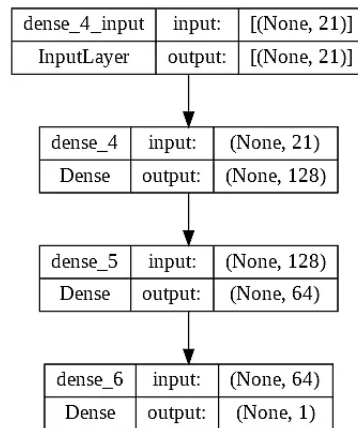


Fig. 2. Proposed Architecture of ANN Model

3.4.6. Bidirectional Long Short Term Memory Model (BiLSTM)

The bidirectional LSTM [33] is a processing architecture that allows the neural network to move data both forward and backwards. Unlike LSTM, BiLSTM is bidirectional, meaning during the forward pass, the model captures data from past to future and in the backward pass, it captures the data from future to past. This helps us preserve data from both past and future. BiLSTMs are used to successfully improve the amount of information available to the algorithm, as well as the context in which that information is provided. Thus, resolves context dependency.

3.4.7. Bidirectional Encoder Representations from Transformers (BERT)

BERT [32] also known as Bidirectional Encoder representations from transformers is a machine learning framework that is currently being used for Natural Language Processing. This framework is based on Transformers, which is a Deep learning model where the weights are calculated based on the connection of every output element to every other input element. BERT's ability to read text input from either left-to-right and right-to-left makes it different from the others and thus, this capability enabled the introduction of transformers, known as bidirectionality. BERT Doesn't require any text pre-processing like TF-IDF vectorization, as it pre-trained on text inputs. Under the Bidirectional capability, BERT is pre-trained on two NLP tasks :

- i) Masked Language Modelling.
- ii) Next Sentence Prediction.

The transformer improves BERT's ability to comprehend context and ambiguity in the provided language. The transformer allows the BERT model to grasp the complete context of the term at once by examining the surrounding words, allowing it to better understand the searcher's purpose. The value of hyperparameters used during our experiment are as listed below: The hyperparameter of dropout layer present in BERT:

- dropout_rate: The value of dropout_rate in our experiment is 0.1.

The hyperparameters of dense layer of BERT are:

- activation : None
- number of dense layers: 1

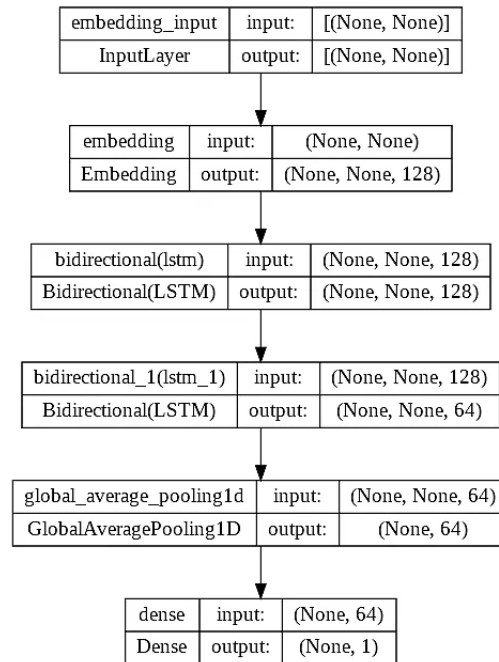


Fig. 3. Proposed Architecture of BiLSTM Model

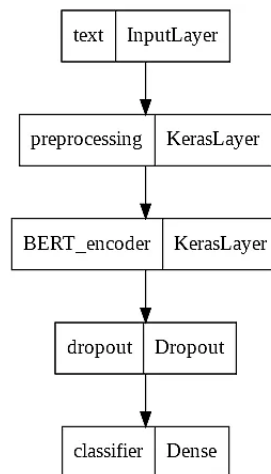


Fig. 4. Proposed Architecture of BERT Model

- name : classifier

Other hyperparameters used are:

- optimizer : Adam- to adjust the learning rate during training
- metric : This is the model evaluation metric, we used BinaryAccuracy.
- loss : The loss function used by us is BinaryCrossEntropy

Hyperparameter introduced by us to handle the classification task:

- threshold : We set the threshold value as 0.5 to classify each record into the respective target class.

4. Metrics Used

4.0.1. Accuracy

Accuracy[34] is one of the evaluation metric that usually describes about the models' performance over all the target classes. This metric can be more useful whenever all the target classes have the same gravity. It can be computed as the ratio of the count of unerring predictions to the combined count of predictions. And is given as follows:

$$Accuracy = \frac{True\ positive + True\ negative}{True\ positive + True\ negative + False\ positive + False\ negative}$$

4.0.2. Precision

The precision[34] is computed as the ratio of the count of Positive specimens unerring classifications to the combined count of specimens labelled as Positive (either correctly or incorrectly). The precision computes the model's accuracy in classifying a test record as positive. If the precision is large, we can trust the model whenever it predicts a test as Positive.

$$Precision = \frac{True\ positive}{True\ positive + False\ positive}$$

4.0.3. Recall

Recall[35] is computed as the ratio of the count of Positive tests precisely allotted as Positive to the combined count of Positive specimens. The recall quantifies the model's ability to ascertain Positive specimens. The higher the recall, the more positive tests are detected. This metric focuses only on how the positive records are classified and is independent of how the negative specimens are classified.

$$Recall = \frac{True\ positive}{True\ positive + False\ negative}$$

4.0.4. F-measure

F-Measure[35] gives us a single score that handles both the problems of recall and precision by providing a single number. Individually, neither recall nor precision gives us the overall story of the model performance. Hence, F-measure is better as it combines the properties of both recall and precision. And it can be computed as follows:

$$F\ measure = \frac{2 * Recall * Precision}{Recall + Precision}$$

4.0.5. Kappa Statistic

Kappa statistic[44] is used to evaluate model classification performance. It measures the degree of agreement among two evaluators, and is popularly referred to as inter-rater reliability. It is computed using the formula given below:

$$Kappa\ Statistic = \frac{O - C}{1 - C}$$

Where "O" is the observed agreement and "C" is the chance agreement

4.0.6. AUC

The AUC score[35] is short for Area Under the Curve. It is computed using Simpson's Rule. The higher the AUC score, the better is our classifier.

$$AUC\ Score = \int TPR\ d(FPR)$$

where, TPR is short for true positive rate and FPR is an abbreviation for false positive rate.

5. Results

The intelligent models (mURLi) help us in the classification task on each of the three datasets used in our work. The NoSQL and SQL problems are binary classification tasks, with classes as "vulnerable" or "benign". The Malicious URL problem is a multi-class classification task resolving to "Spam", "Phishing", "Malware", "Defacement" and "Benign".

We noticed that all the datasets used, were imbalanced, and this skewness hinders the model learning, as the model can be biased towards a particular class. To tackle this issue, we oversampled using SMOTE (synthetic minority oversampling technique) to make the ratio of all the target classes equal for training.

Table 1. Before and after Oversampling

Sl no.	Dataset	Before Oversampling	After Oversampling
1	Malicious URL	14236	19955
2	NoSQL	753	1226
3	SQL	3360	4936

We used Random Forest, KNN, AdaBoost, XGBoost, Artificial Neural Network, BiLSTM and BERT for NoSQL injection as well as SQL Injection to obtain a binary value, the same models are used for malicious URLs dataset to get a multi-class prediction. This was done in Google Colaboratory's simulated Python environment using Nvidia K80/ T4 GPUs.

The table below lists the scores for all the models used, each being tested using different performance metrics. From the results, we infer two models that can be used for the detection tasks, one being a machine learning model and the other one is a deep learning model. It is evident from the results that Malicious URLs are best identified using Random forest and BiLSTM model, with an accuracy of 95.04% and 94.41% respectively. NoSQL injection attacks are best detected by the Random Forest and BERT model at 94.82% and 99.01% accuracy, respectively. And lastly for the task of SQLi the best suited models are Random Forest, ANN and BERT with an accuracy of 97.26%, 97.14% and 99.84% respectively.

Another notable inference from the results is the reason behind the poor performance of BiLSTM model for NoSQLi, and consistent best performance of BERT model. The poor performance of BiLSTM model is attributed to the lack of large training dataset, and the consistent high accuracy of BERT model is attributed to its pre-training.

To help analyze and compare the performance of each model, we have used line graphs. We used Matplotlib python library for line graphs and this graph is plotted using the scores tabulated in Fig.5.

Dataset	Classifiers	Accuracy	Precision	Recall	F1 Score	Kappa Score	ROC AUC Score
Malicious URL	XGBoost	94.21	94.25	94.20	94.21	92.60	99.47
	Ada Boost	82.36	84.13	82.36	82.83	77.52	93.33
	KNN	90.47	90.67	90.47	90.49	87.86	97.62
	Random Forest	95.04	95.08	95.04	95.03	93.68	99.29
	ANN	90.28	90.97	90.28	90.25	87.63	96.27
	BiLSTM	94.41	94.68	94.41	94.40	92.88	97.51
NoSQL	XGBoost	93.63	88.46	82.14	85.19	81.13	94.50
	Ada Boost	92.43	83.64	82.14	82.88	78.02	94.23
	KNN	83.67	53.73	78.26	63.72	53.60	78.02
	Random Forest	94.82	93.88	82.14	87.62	84.36	93.20
	ANN	82.47	55.56	21.74	31.25	25.25	79.04
	BiLSTM	81.67	50.00	86.96	63.49	52.42	83.72
	BERT	99.01	94.74	100	97.30	96.69	99.40
SQL	XGBoost	96.90	95.65	93.22	94.42	92.28	99.15
	Ada Boost	95.71	90.98	94.07	92.50	89.50	99.34
	KNN	95.71	92.37	92.37	92.37	89.39	97.69
	Random Forest	97.26	96.51	93.64	95.05	93.16	99.0
	ANN	97.14	96.49	93.22	94.83	92.85	95.35
	BiLSTM	92.80	87.67	86.52	87.09	82.10	90.88
	BERT	99.84	99.91	99.66	99.79	99.65	99.80

Fig. 5. Results

The following graphs depict the accuracy followed by the kappa-statistic scores of all the models used for respective tasks. The blue line graphs illustrate the accuracy values and the orange line graphs illustrate kappa-scores.

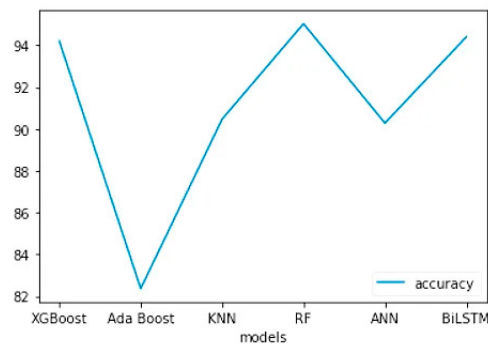


Fig. 6. Accuracy scores for Malicious URL detection

6. Conclusion and Future Scope

The key takeaway of our research is the importance of feature engineering and balanced dataset. Inspired by the feature extraction done by prevailing work, especially in detection of NoSQLi, we went ahead with an extensive

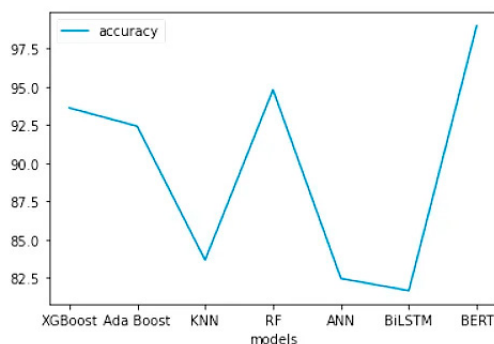


Fig. 7. Accuracy scores for NoSQLi detection

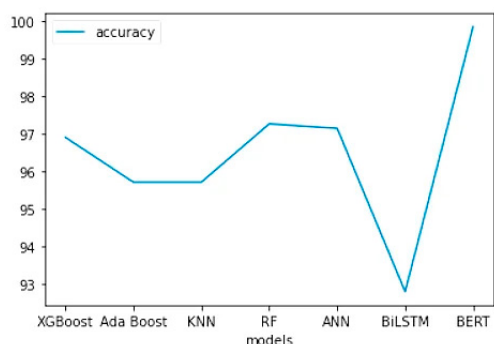


Fig. 8. Accuracy scores for SQLi detection

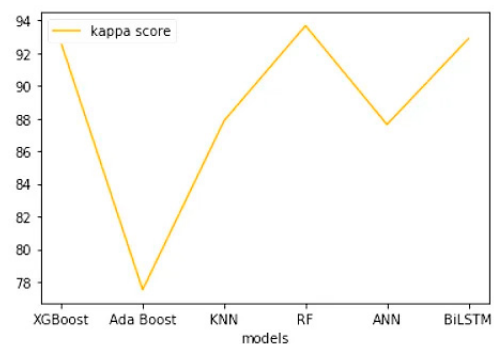


Fig. 9. Kappa scores for Malicious URL detection

feature extraction for both SQLi and NoSQLi. The strong features for SQL injection attacks with a dataset large enough to achieve unbiased training of models helped us achieve the best results as listed in the table. The accuracy of NoSQLi is not as high as that of SQLi, even after extensive feature extraction, this is due to fewer records in the training set.

Drawing inferences from the experimental procedure and the results, we see that BERT works well for text-only datasets, which includes the NoSQL and SQL dataset. It doesn't give impressive results on malicious URLs, as it is a numeric dataset. The prevailing research did not explore much of deep learning models for the task at hand, so our research work experiments with the usage of three deep learning models namely, BERT, BiLSTM and ANN, and they outperformed few of the Machine learning models and BERT achieved the best accuracy for the task of injection attack detection while BiLSTM gave the best results for the detection of malicious URLs. The proposed tool, thus,

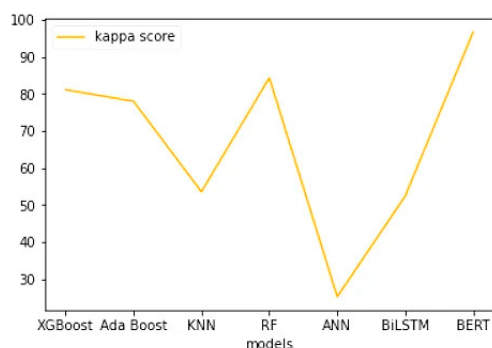


Fig. 10. Kappa scores for NoSQLi detection

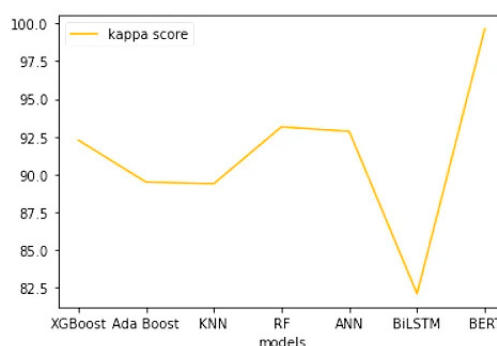


Fig. 11. Kappa scores for SQLi detection

solves three major threats by helping in detecting them as early as possible with a large accuracy.

The future scope of our project is to come up with a better and much larger dataset, especially for NoSQL injection attacks, to improve on the model training. And further experiment with the combination of various deep learning models to ponder on the results.

References

- [1] CSO Online, SQL injection Attacks Led to Heartland, Hannaford Breaches, <https://www.csoonline.com/article/2124293/identity-theft-prevention-sql-injection-attacks-led-to-heartland-hannaford-breaches.html>
- [2] Freepik data breach: Hackers stole 8.3M records via SQL injection, <https://www.bleepingcomputer.com/news/security/freepik-data-breach-hackers-stole-83m-records-via-sql-injection/>
- [3] MongoDB, “Nosql databases explained,” <https://www.mongodb.com/nosql-explained>.
- [4] NoSQL injection bugs in Rocket.Chat left servers open to RCE attacks, <https://portswigger.net/daily-swig/nosql-injection-bugs-in-rocket-chat-left-servers-open-to-rce-attacks>
- [5] 150 Million Users Records From Iranian Raychat App Leaked Online, <https://www.hacktoday.com/150-million-users-records-from-iranian-raychat-app-leaked-online/>
- [6] Mohammad Saiful Islam Mamun, Mohammad Ahmad Rathore, Arash Habibi Lashkari, Natalia Stakhanova and Ali A. Ghorbani, “Detecting Malicious URLs Using Lexical Analysis”, Network and System Security, Springer International Publishing, P467–482, 2016.
- [7] SYED SAQLAIN HUSSAIN SHAH, SQL Injection Dataset Kaggle, <https://www.kaggle.com/datasets/syedsaqlainhussain/sql-injection-dataset>
- [8] <https://www.ibm.com/in-en/topics/knn>
- [9] Aha, D.W. et al. Instance-based learning algorithms. , Machine Learning, 6. (1991) , 37–66
- [10] Jin, C. et al. An improved ID3 decision tree algorithm. , 2009 4th International Conference on Computer Science & Education. (2009)
- [11] Hopfield, J.J. Artificial neural networks. , IEEE Circuits and Devices Magazine, 4. (1988) , 3–10
- [12] Ho, T.K. Random decision forests. , Proceedings of 3rd International Conference on Document Analysis and Recognition

- [13] Freund, Y. and Schapire, R.E. A decision-theoretic generalization of on-line learning and an application to boosting. , Lecture Notes in Computer Science. (1995) , 23–37
- [14] M., A. et al. NoSQL Racket: A Testing Tool for Detecting NoSQL Injection Attacks in Web Applications. , International Journal of Advanced Computer Science and Applications, 8. (2017)
- [15] Chen, T. and Guestrin, C. XGBoost. , Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. (2016)
- [16] Joseph, S. and Jevitha, K.P. An Automata Based Approach for the Prevention of NoSQL Injections. , Communications in Computer and Information Science. (2015) , 538–546
- [17] Wichers, D.: OWASP Top Ten Project. <https://owasp.org/www-project-top-ten/> Online; accessed March 2021.
- [18] <https://beaglesecurity.com/blog/article/server-side-request-forgery-attack.html>
- [19] Server-Side Request Forgery (SSRF) & the Cloud Resurgence, (2020), accessed 2021, <https://appcheck-ng.com/server-side-request-forgery-ssrf/>
- [20] Islam, Md Rafid Ul, et al. "Automatic detection of nosql injection using supervised learning." 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC). Vol. 1. IEEE, 2019.
- [21] Shachi, Mehjabeen, et al. "A Survey on Detection and Prevention of SQL and NoSQL Injection Attack on Server-side Applications." International Journal of Computer Applications 975: 8887.
- [22] Kim, M.-Y. and Lee, D.H. Data-mining based SQL injection attack detection using internal query trees. , Expert Systems with Applications, 41. (2014) , 5416–5430
- [23] Lashkaripour, Z. and Ghaemi Bafghi, A. A security analysis tool for web application reinforcement against SQL injection attacks (SQLIAs). , 2013 10th International ISC Conference on Information Security and Cryptology (ISCISC). (2013)
- [24] G. and Pathak, K. SQL Injection Attacks: Technique and Prevention Mechanism. , International Journal of Computer Applications, 69. (2013) , 35–39
- [25] Website. . [Online]. Available: D. Box and A. Hejlsberg. LINQ: .NET Language-Integrated Query. <https://msdn.microsoft.com/en-us/library/bb308959.aspx>. [Accessed: 04-May-2021]
- [26] Lee, I. et al. A novel method for SQL injection attack detection based on removing SQL query attribute values. , Mathematical and Computer Modelling, 55. (2012) , 58–68
- [27] Moore, A.W. and Lee, M.S. Efficient Algorithms for Minimizing Cross Validation Error. , Machine Learning Proceedings 1994. (1994) , 190–198
- [28] Ross Quinlan, J. (1993) C4.5: Programs for Machine Learning, Morgan Kaufmann.
- [29] cr0hn, "nosqlinjection_wordlists", https://github.com/cr0hn/nosqlinjection_wordlists
- [30] <https://towardsdatascience.com/https-medium-com-vishalmorde-xgboost-algorithm-long-she-may-rein-edd9f99be63d>
- [31] <https://www.sciencedirect.com/topics/engineering/adaboost>
- [32] Devlin, Jacob, et al. "Bert: Pre-training of deep bidirectional transformers for language understanding." arXiv preprint arXiv:1810.04805 (2018).
- [33] Brahma, Siddhartha. "Improved sentence modeling using suffix bidirectional lstm." arXiv preprint arXiv:1805.07340 (2018).
- [34] ISO, ISO. "5725-1: 1994, Accuracy (trueness and precision) of measurement methods and results-Part 1: General principles and definitions." International Organization for Standardization, Geneva (1994).
- [35] Powers, David MW. "Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation." arXiv preprint arXiv:2010.16061 (2020).
- [36] <https://github.com/payloadbox/sql-injection-payload-list>
- [37] Al-talak, Khadejah, and Onytra Abbass. "Detecting Server-Side Request Forgery (SSRF) Attack by using Deep Learning Techniques." International Journal of Advanced Computer Science and Applications 12.12 (2021).
- [38] Ho, Tin Kam. "Random decision forests." Proceedings of 3rd international conference on document analysis and recognition. Vol. 1. IEEE, 1995.
- [39] Fix, Evelyn, and Joseph Lawson Hodges. "Discriminatory analysis. Nonparametric discrimination: Consistency properties." International Statistical Review/Revue Internationale de Statistique 57.3 (1989): 238-247.
- [40] Chen, Tianqi, and Carlos Guestrin. "Xgboost: A scalable tree boosting system." Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining. 2016.
- [41] Freund, Yoav, and Robert E. Schapire. "A decision-theoretic generalization of on-line learning and an application to boosting." Journal of computer and system sciences 55.1 (1997): 119-139.
- [42] Hastie, Trevor, et al. "Multi-class adaboost." Statistics and its Interface 2.3 (2009): 349-360.
- [43] Rosenblatt, Frank. "The perceptron: a probabilistic model for information storage and organization in the brain." Psychological review 65.6 (1958): 386.
- [44] McHugh, Mary L. "Interrater reliability: the kappa statistic." Biochemia medica 22.3 (2012): 276-282.